

MSc Data Science Project

7PAM2002

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Exploring Renewable Energy and Energy Efficiency Impacts on
Sectoral Economic Growth in Low- and Middle-Income Countries.

Student Name and SRN:

Nazmul Hossain, 23015862

Supervisor: William Bate

Date Submitted: Enter the date you are submitting this report

Word Count: Enter the word count excluding references and
appendices

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science **in Data Science** at the University of Hertfordshire.

I have read the detailed guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project or course.

I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6)

I did not use human participants in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

Student Name printed: Nazmul Hossain

Student Name signature

Student SRN number: 23015862

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Table of Contents

DECLARATION STATEMENT.....	2
----------------------------	---

1. Introduction

2. Background / Literature Comparison

3. Dataset

4. Ethical Issues

5. Methodology

5.1 Tools, Libraries, and Environment

To conduct this comprehensive analysis, I worked extensively in **Python**, alternating between **Jupyter Notebook** for iterative local development and **Google Colab** for resource-intensive tasks such as rendering large heatmaps and geospatial plots. This hybrid setup allowed me to optimize both speed and flexibility depending on the computational needs of each task.

I employed a wide suite of powerful Python libraries, each carefully selected based on their capabilities in data science and machine learning:

- **Pandas** and **NumPy** served as my core tools for managing and transforming large-scale, multi-year, multi-indicator time-series data. I used them for reshaping, merging, missing value handling, and statistical computations.
- **Matplotlib** and **Seaborn** were used for crafting an extensive range of visualizations — including box plots, violin plots, multi-type time series trends, histograms, correlation heatmaps, PCA scree plots, and clustering diagrams. I relied on Seaborn's expressive color palettes (e.g., "coolwarm", "Set2") for visual clarity.
- **Scikit-learn** (sklearn) was the backbone of my **machine learning workflow**. I used it for **data standardization (StandardScaler)**, **dimensionality reduction (PCA)**, **unsupervised learning (KMeans clustering)**, and **internal validation metrics (silhouette score)**.
- For **geospatial visualization**, I turned to **GeoPandas** in combination with **Natural Earth shapefiles** (ne_110m_admin_0_countries.shp). This enabled me to map country clusters geographically and observe regional patterns that transcended income level boundaries.

Regarding data sources, I used **two distinct datasets**:

- The **primary dataset** was a World Bank-derived CSV file containing 91 economic, energy, and environmental indicators per country from **1989 to 2023**. This dataset was pre-structured into columns such as "Series Name", "Country Code", and yearly values (e.g., "1990 [YR1990]", "2023 [YR2023]"), which I then cleaned and transformed for analysis.
- The **income classification dataset** was retrieved from the official World Bank classification portal via the URL:
<https://databank.worldbank.org/data/download/site-content/CLASS.xlsx>.
This Excel sheet contained yearly income categorizations such as "Low income", "Lower middle income", and "Upper middle income" for all recognized countries.

In total, this environment and toolchain enabled me to maintain a consistent, modular, and highly replicable analytical pipeline from preprocessing through to final clustering and visualization stages.

5.2 Data Preprocessing

To prepare the dataset for rigorous, income-level-based analysis, I undertook a series of highly structured preprocessing steps. I combined technical precision with logical checks to ensure the dataset was reliable, standardized, and robust for all downstream statistical and machine learning procedures.

I began by importing essential libraries — NumPy, Pandas, and Matplotlib — to facilitate numerical operations, dataframe manipulation, and initial inspections. I loaded the raw dataset from the World Bank repository using the `.read_csv()` function and immediately addressed the issue of placeholder values ("..") by replacing them with `pd.NA`, thereby marking them as true missing values in Pandas.

Next, I isolated only the year-specific columns starting from the 5th column onwards using `df.columns[4:]`, and applied `pd.to_numeric()` with `errors='coerce'` to convert all these values into float-compatible formats. This ensured consistent numerical interpretation of values across time, coercing any remaining anomalies into `NaN`.

To impute these missing values meaningfully, I applied **group-wise forward and backward filling** — first by grouping the dataframe by ["Country Name", "Series Name"] and then transforming the year columns using a chained `.bfill().ffill()` method. This technique maintained the integrity of time-series continuity, ensuring that missing values within a country's series were filled only using that same country-series context.

While working on post-fill quality, I noticed some anomalous values in the 2023 column. To avoid inflated values corrupting my trend analysis, I inserted a rule-based filter where any 2023 value greater than 1.5× of its corresponding 2022 value was replaced with the 2022 value. This step was implemented using the `.where()` method:

```
df["2023 [YR2023]"] = df["2023 [YR2023]"].where(df["2023 [YR2023]"] < df["2022 [YR2022]"] * 1.5, df["2022 [YR2022]"])
```

Following this, any remaining `NaN` values were imputed using the **row-wise mean of each country-series pair**, implemented via `.apply(lambda row: row.fillna(row.mean()), axis=1)`. This strategy preserved the contextual distribution of each individual row rather than borrowing information across rows or columns.

To avoid distortions from sparsely populated series, I calculated a missing-value threshold set at **50% of the number of year columns**. Any row (i.e., country-indicator pair) with missing values exceeding this threshold was dropped entirely using `df.dropna(thresh=missing_threshold, axis=0)`, ensuring that only meaningful rows were retained for downstream analysis.

Once my base dataset was cleaned, I moved to merge it with the **World Bank's income classification dataset** for 2024–2025, sourced from:

<https://databank.worldbank.org/data/download/site-content/CLASS.xlsx>.

I used `pd.read_excel()` with `skiprows=3` and custom column names to ingest the sheet titled "**List of economies**", extracting only ["Country Name", "Country Code", "Income Level"] for the merge. Using an inner join on both "Country Name" and "Country Code" allowed me to accurately map each country to its respective income group.

After the merge, I identified unmatched records — i.e., countries with NaN in "Income Level" — using logical filtering. Many of these were **regions or aggregates**, not actual countries, such as "World", "Euro area", or "Sub-Saharan Africa". I compiled a comprehensive list of these aggregate entries and removed them using `~df["Country Name"].isin(regions_to_remove)`. This filtered the dataset to country-level rows only.

Still, a handful of actual countries remained unmatched due to naming mismatches or edge-case classifications. I reviewed these manually and constructed a mapping dictionary of corrected "Income Level" values using domain knowledge and World Bank classification updates. For instance:

```
manual_income_levels = {
    "Cote d'Ivoire": "Lower middle income",
    "Sao Tome and Principe": "Lower middle income",
    "Turkiye": "Upper middle income",
    "Viet Nam": "Lower middle income",
    "Afghanistan": "Low income",
    "Venezuela, RB": "Not classified",
    ...
}
```

I applied this mapping using `.map()` and `.fillna()` to ensure a fully populated "Income Level" column. Next, I **explicitly excluded** high-income countries and "Not classified" entries using a final logical filter. This brought the dataset down to only three groups: **Low income**, **Lower middle income**, and **Upper middle income**.

To validate data quality, I conducted an assertive check for duplicates using `.duplicated().sum()` and confirmed that no duplicates remained. Additionally, I verified that the "Income Level" column contained only the three intended classifications via an assertion check:

```
assert all(income in valid_income_levels for income in unique_income_levels)
```

As a final step, I incorporated expert recommendation updates. For instance, I removed "Bulgaria", "Palau", and "Russian Federation" manually, as these countries had just transitioned to the high-income bracket (World Bank, 2025). This filtering ensured temporal consistency between our data and the latest World Bank classification.

I then saved the fully processed dataset to disk using `.to_csv()`, ensuring reproducibility. From this point forward, the resulting dataset was clean, structured, and enriched with

income group metadata — making it ready for exploratory, statistical, and machine learning-based analyses

5.3 Exploratory Data Analysis (EDA)

After preparing the dataset, I transitioned into exploratory analysis to derive meaningful insights. My EDA process was comprehensive and multi-layered, spanning descriptive statistics, visual exploration, time-series tracking, and correlation analysis. I ran the analysis using a combination of **Pandas**, **Seaborn**, **Matplotlib**, and **NumPy**, all within the Jupyter environment.

Visualizing Country Distribution by Income Levels

To begin with, I performed a categorical breakdown of countries by income level using a **pie chart**. I created a new dataframe (`unique_countries_df`) by dropping duplicate country names, ensuring each country was represented only once. This allowed me to visualize the **proportion of countries** across the three target income levels — Low, Lower-middle, and Upper-middle — using `value_counts()` and `.plot.pie()` on the “Income Level” column. The chart helped confirm the dataset's coverage and ensured income levels were adequately represented.

Analyzing 2023 Indicator Values Across Income Levels

Next, I analyzed the distribution of 2023 values (across all indicators) for each income level. Using `sns.boxplot()` and `sns.violinplot()`, I explored both central tendencies and dispersion patterns. These **box plots** and **violin plots** provided insights into data spread, skewness, and outliers in each income group. In addition to this, I created **income-specific histograms** using `sns.histplot()` (with KDE enabled) to visualize the **frequency distribution** of 2023 values. This allowed me to detect which income groups had more consistent versus volatile performances.

Global GDP Growth Trend Analysis

To understand macroeconomic behavior, I isolated the series “**GDP growth (annual %)**” from the dataset. I dropped the income level column temporarily to focus on global averages. Then, I calculated the **mean GDP growth across all countries** for each year using `mean()` on the subset's year columns. I visualized this trend using a **line plot**, adding markers and gridlines for readability. The resulting plot clearly revealed fluctuations in global GDP growth over time and established a macroeconomic timeline against which I could compare emissions and energy use.

Trend Analysis of CO₂ Emissions Indicators Across Income Levels

The next phase involved a deep dive into **CO₂ emissions indicators**. I created three custom visualization functions — `plot_emission_line()`, `plot_emission_bar()`, and `plot_emission_box()` — to automate time-series, aggregated, and distributional plots respectively.

I supplied these functions with:

- A `series_name` (e.g., *Carbon dioxide (CO₂) emissions from Power Industry*)
- An `income_level` (e.g., *Low income*)

Inside each function:

- I **filtered** the dataset using boolean masking
- I **converted year columns to numeric** using `pd.to_numeric(errors="coerce")`
- In line plots, I iterated over each country's row and plotted all countries' trends using a loop with `plt.plot()`
- In bar plots, I aggregated yearly means using `mean().reset_index()` and visualized them using `sns.barplot()` with color palettes and legends
- In box plots, I transposed the dataframe to visualize distributions per year using `sns.boxplot()`

I looped through **9 carefully selected CO₂ emissions series** (e.g., total emissions, per capita emissions, sector-specific emissions, carbon intensity of GDP, etc.) for **each of the three income groups**. This gave me 81 plots in total and a rich set of visual trends to interpret.

Thematic Correlation Analysis: Linking All Four Research Dimensions

To directly address my research question, I compiled four indicator blocks:

1. **Renewable energy adoption**
2. **Energy efficiency**
3. **Sectoral economic growth**
4. **Emissions and carbon impact**

I manually curated lists of relevant series under each block (e.g., *Renewable energy consumption (% of total final energy consumption)*, *Energy use per \$1,000 GDP*, *Services value added growth*, *Total greenhouse gas emissions excluding LULUCF*, etc.). These were combined into one master list: `all_research_series`.

Income-Based Correlation Analysis

Using the series from the research blocks, I built a function `plot_correlation_by_income()` that:

- Filtered rows based on income level and relevant series
- Converted year columns to numeric
- Computed a row-wise average for each indicator per country (`Series Avg`)
- Pivoted the dataframe using `pivot_table()` so each row represented a country, and each column an indicator

For each income level (**Low**, **Lower-middle**, and **Upper-middle**), I computed **three types of correlation matrices**:

- **Pearson correlation** (`.corr(method='pearson')`) – capturing linear relationships
- **Spearman correlation** (`.corr(method='spearman')`) – capturing monotonic (rank-based) relationships
- **Kendall correlation** (`.corr(method='kendall')`) – capturing ordinal concordance relationships

I visualized these using `sns.heatmap()` with annotations and styling, yielding **three separate heatmaps per income level**. These plots allowed me to evaluate how energy use, emissions, and economic growth indicators co-varied under different assumptions about their relationships. Comparing results across methods revealed both linear and nonlinear dynamics, enhancing analytical robustness.

Time-Based Correlation Analysis (1990–2005 vs 2006–2023)

To trace how indicator relationships evolved over time, I divided the time series into two intervals:

- **Early Years**: 1990 to 2005
- **Recent Years**: 2006 to 2023

Using slicing logic on column names, I calculated two new columns:

- `Early Avg (1990–2005)`
- `Recent Avg (2006–2023)`

I used `pivot()` to generate wide-format tables for each income level and time period. On each of these, I computed three separate correlation matrices using Pearson, Spearman, and Kendall methods. I visualized them using `plot_correlation_heatmap()` for a total of **six heatmaps per income level** — enabling historical comparison of not just correlation strength, but also changes in the nature (linear, monotonic, ordinal) of those relationships over decades.

Sector-Based Correlation Analysis

Lastly, I explored how **CO₂ emissions, energy use, and renewables** influenced **sectoral growth** in **Agriculture, Industry, and Services**. I defined a dictionary `sector_series_dict` and looped over each income level and sector.

For each combination, I:

- Filtered the dataset by income and selected sector-relevant indicators
- Averaged year-wise values to derive `Series Avg`
- Created a country-series pivot table using `pivot_table()`
- Computed **three different correlation matrices** — Pearson, Spearman, and Kendall — for each sector
- Visualized each matrix using `sns.heatmap()` with customized formatting

This yielded a total of **27 heatmaps** (3 sectors × 3 income levels × 3 correlation types). Combinations with insufficient data (`pivot_df.shape[0] < 3`) were automatically skipped to preserve statistical integrity. The multi-method approach helped me triangulate sectoral insights and reduce the risk of over-reliance on a single correlation paradigm.

This exhaustive EDA served not only as the diagnostic phase of my project but also laid the groundwork for dimensionality reduction and cluster-based grouping in later stages. It helped me identify structural similarities across countries, changes over time, and thematic patterns across income classes.

5.4 Principal Component Analysis (PCA)

Once I had established thematic patterns from EDA, I transitioned into **dimensionality reduction** using **Principal Component Analysis (PCA)** to reveal latent structures and compress multivariate relationships across income levels. I handled PCA separately for **Low**, **Lower-middle**, and **Upper-middle** income countries, treating them as **independent data universes**.

To begin, I manually compiled a list of 17 indicators by merging four blocks—**renewable energy, energy efficiency, sectoral economic growth**, and **CO₂ emissions**—that mapped to the four core themes of my research question. I selected these series to capture a holistic yet compact representation of energy-growth-emission interdependencies.

For each income group:

- I filtered the master dataset to retain only countries from the target income level and only rows where `Series Name` belonged to `selected_series`.
- Next, I **converted all year columns to numeric**, dropping any parsing-related issues using `errors='coerce'`, ensuring every series was analysis-ready.

- I computed a **row-wise mean across all years** to derive a time-agnostic column called "Series Avg" for every indicator-country pair, using `.mean(axis=1)`.
- Using `pivot_table()`, I transformed the dataframe so that each **row represented a country**, and each **column a research-relevant indicator**. I excluded countries that had missing data across any of the selected series by applying `.dropna()`.

Then, I applied **standardization** using `StandardScaler()` from scikit-learn to ensure each series was centered and scaled (mean=0, variance=1). This step was crucial to prevent high-magnitude indicators from dominating the PCA components.

I fit PCA on the scaled matrix with `PCA().fit_transform()` to reduce the dataset into a new **orthogonal coordinate system**. From this:

- I captured **explained variance ratios** of each principal component
- I generated a **scree plot** showing the cumulative variance explained, helping visualize how much information could be preserved using only two components

Finally, I constructed a **two-dimensional PCA scatter plot**, showing countries in the new PC1–PC2 space using `sns.scatterplot()`. I called `plot_pca_with_labels()` to annotate each point with its country name, enhancing interpretability.

After generating PCA plots, I aimed to go beyond visual clustering by **quantifying inter-country similarity**. For this, I computed a **Euclidean distance matrix** between countries using the original PCA input space (i.e., standardized Series Avg vectors). I implemented this through `pdist()` and converted it into a squareform matrix.

To enhance visual clarity and interactivity, I created an **interactive Plotly heatmap** using `px.imshow()`, which displayed country-wise distances inside each heatmap cell and allowed zooming and panning. These plots were saved in HTML format for documentation and easy sharing.

Through this, I uncovered patterns of intra-income group heterogeneity — for instance, while some Low-income countries clustered tightly (e.g., Chad and Congo Dem. Rep.), others like Ethiopia or Afghanistan appeared distant.

This helped me realize that not all countries in the same income level behave the same way, which challenges the assumptions behind income-level grouping and justifies my use of PCA and clustering.

5.5 Clustering and Profile Analysis

Building on the PCA foundation, I conducted **unsupervised clustering** using the **KMeans algorithm** on the PCA coordinates for each income group. I designed a function `run_kmeans_clustering()` that:

- Accepted PCA coordinates and maximum k (default 12)
- Iteratively ran KMeans from **k = 2 to optimal**, storing **inertia** and **silhouette scores**

- Chose optimal k using the **maximum silhouette score**, then fit a final KMeans model to label each country

I visualized the **elbow curve** to show inertia reduction per cluster count, helping support the chosen k. Then, I plotted the final cluster results in the PC1–PC2 space using **cluster-wise scatter plots** with text annotations for each country. These helped visually evaluate clustering separation and density.

To understand what defined each cluster, I profiled them using the original (non-PCA) indicators:

- I reloaded the original indicators per country using a filter on Income Level and countries used in PCA
- I grouped by cluster label and calculated **cluster-wise mean values** for all indicators
- I normalized those averages using **min–max scaling**, and generated **radar-like line plots** to contrast profiles across clusters visually

Next, I implemented **geographic mapping of clusters** using GeoPandas. I downloaded the **Natural Earth shapefile** for country boundaries and merged it with my cluster data. I corrected known naming mismatches between World Bank names and shapefile names using a manual `rename_map` dictionary. Then:

- I plotted each country shaded by cluster assignment
- I annotated maps with **country names using centroid coordinates**, keeping label sizes readable
- I repeated this for all income levels, generating three separate maps

Finally, to facilitate comparative interpretation, I wrote a composite function `plot_all_income_levels_geoclusters()` that:

- Mapped the **clustered countries side-by-side** using `subplots(1, 3)`
- Harmonized plotting scale, colouring, and legend handling
- Created a **visual dashboard of global clustering outcomes** by income level

This multifaceted clustering approach allowed me to:

- Identify **latent country groupings** beyond income level
- Profile each group in terms of indicator performance
- Visualize global **geospatial patterns** in energy-growth-emission profiles

Together, PCA and KMeans uncovered **hidden structures** that would have been lost in standard analysis, making them essential for the interpretative depth of this project.

Here is your full **Panel Regression Methodology** section, written inline in the same exhaustive, technical, and personal style as your previous methodology parts:

5.6 Panel Regression Methodology

To rigorously examine the sectoral economic growth drivers across income groups, I employed panel data regression techniques using both Fixed Effects and Random Effects models. My goal was to quantify the influence of selected energy, emissions, and infrastructure indicators on the growth of agriculture, industry, and services sectors.

Library and Environment Setup

I used Python in a Jupyter environment, leveraging the `pandas` and `numpy` libraries for data manipulation, and `linearmodels` for estimating panel regressions. Additionally, I imported `statsmodels.api.add_constant` for including intercept terms, and `scipy.stats` for executing the Hausman test.

```
from linearmodels.panel import PanelOLS, RandomEffects
from statsmodels.tools.tools import add_constant
from scipy import stats
```

Selection of Target Variables

I focused on three key sectoral growth indicators from the World Bank:

- **Agriculture, forestry, and fishing, value added (annual % growth)**
- **Industry (including construction), value added (annual % growth)**
- **Services, value added (annual % growth)**

Each served as a dependent variable in a separate model. For each sector, I executed regressions individually for **low-income**, **lower-middle-income**, and **upper-middle-income** countries.

Independent Variable Selection

I selected the following five predictors based on domain knowledge, EDA, and correlation analysis:

1. Renewable energy consumption (% of total final energy consumption)
2. Energy intensity level of primary energy (MJ/\$2017 PPP GDP)
3. Access to electricity (% of population)
4. GDP per unit of energy use (PPP \$ per kg of oil equivalent)
5. Carbon dioxide (CO₂) emissions excluding LULUCF per capita (t CO₂e/capita)

These features were consistently used across all sectoral models. I later experimented with adding GDP growth (annual %) and PM2.5 air pollution based on their potential relevance and availability.

Data Preparation

I defined a reusable function `prepare_panel_data()` that extracted and reshaped the panel dataset for a given income level and dependent variable. This function:

- Filtered countries by income group
- Selected only relevant series names
- Melted the dataset from wide to long format
- Pivoted it to ensure each row represented a (country, year) pair
- Dropped missing values
- Converted year to integer

```
df_long = df_filtered.melt(id_vars=['Country Name', 'Series Name'],  
var_name='Year', value_name='Value')  
df_panel = df_long.pivot_table(index=['Country Name', 'Year'],  
columns='Series Name', values='Value').reset_index()
```

I renamed columns for clarity and ensured the dataset had no nulls before regression.

Regression Execution

For each prepared dataset (e.g., `agri_df_low`), I used `PanelOLS` to fit the **Fixed Effects** model and `RandomEffects` to estimate the **Random Effects** model. Both models were fitted with **robust standard errors** to ensure valid inference under heteroskedasticity.

```
fe_model = PanelOLS(y, X,  
entity_effects=True).fit(cov_type='robust')  
re_model = RandomEffects(y, X).fit(cov_type='robust')
```

Hausman Test and Model Choice

To determine whether the Fixed or Random Effects model was more appropriate, I implemented the **Hausman test** manually. I computed the difference in coefficients and their covariance matrices to derive the test statistic and p-value.

```
b_diff = fe_results.params - re_results.params  
cov_diff = fe_results.cov - re_results.cov
```

```
stat = b_diff.T @ np.linalg.inv(cov_diff) @ b_diff
```

If the p-value was below 0.05, I preferred the Fixed Effects model; otherwise, I used Random Effects. The model selection logic and summaries were printed for transparency.

Model Evaluation and Visualization

After model fitting, I extracted the predicted values and added them back to the original dataset for comparison. I then plotted the **actual vs predicted growth** for one representative country per income group, using matplotlib.

```
plt.plot(df_sample['year'], df_sample['actual'], label='Actual',  
marker='o')  
plt.plot(df_sample['year'], df_sample['predicted'],  
label='Predicted', marker='x')
```

PCA-based Validation

As a final methodological step, I conducted robustness validation by restricting the dataset to countries selected through prior Principal Component Analysis (PCA). These countries were identified as structurally representative within their income group based on energy-emission-growth profiles.

To implement this:

- I loaded PCA coordinate files for each income group.
- Then I filtered the main dataset to retain only those countries present in the PCA coordinate set.
- Using the filtered subset, I reconstructed the panel datasets and reran the same fixed and random effects models for a single sector and income group as a **targeted diagnostic test**, keeping all variable selections and model specifications unchanged.

6. Results

7. Analysis & Discussion

8. Conclusion

9. References

10. Appendices